

Chapter 6, Section A: Function Declarations vs Function Expressions

1. Function Declaration

```
function greet(name) {  
  return `Hello, ${name}`;  
}
```

- Starts with the function keyword, followed directly by a name.
 - Not assigned to a variable - it exists on its own.
 - No semicolon needed after the closing }.
-

2. Function Expression

```
const greet = function (name) {  
  return `Hello, ${name}`;  
};
```

- A function created as a **value**, then assigned to a variable (const, let, or var).
- The function itself can be **named** or **anonymous**. The example above is anonymous (no name after function).
- A semicolon is needed at the end, because this is an assignment statement, same as const x = 5;.

Named function expression (less common, but valid):

```
const greet = function sayHello(name) {  
  return `Hello, ${name}`;  
};
```

The inner name sayHello is only usable **inside** the function itself (e.g. for recursion) - outside, you still call it via greet.

3. The key difference: hoisting

Hoisting means JavaScript moves declarations to the top of their scope before running the code.

Feature	Declaration	Expression
Hoisted?	Yes - fully, including the function body	Only the variable is hoisted (if var), not the function itself
Can call before its line in the code?	Yes	No - throws an error

Declaration - works even before it's written

```
sayHi(); // "hi" -- works fine
```

```
function sayHi() {  
  console.log("hi");  
}
```

JavaScript hoists the **entire function**, so it is fully usable from the very top of the scope.

Expression with var - hoisted as undefined, not as a function

```
sayBye(); // TypeError: sayBye is not a function  
  
var sayBye = function () {  
  console.log("bye");  
};
```

With var, the **variable** sayBye is hoisted (so it exists early), but its value at that point is undefined, not a function. Calling undefined() throws a TypeError.

Expression with let/const - not usable at all before the line

```
sayHello(); // ReferenceError: Cannot access 'sayHello' before  
initialization  
  
const sayHello = function () {  
  console.log("hello");  
};
```

let and const variables are hoisted but stay in the **Temporal Dead Zone (TDZ)** until their line runs - this connects to what you already know about let/const hoisting from Chapter 2.

4. Why this matters in practice

- Function declarations let you organize code with “helper” functions defined **below** where they’re used, which many codebases prefer for readability.
- Function expressions are required when you want to:
 - assign a function conditionally
 - pass a function immediately as a value (e.g. as a callback)
 - control exactly when the function becomes available

```
let operation;  
  
if (useMetric) {  
  operation = function (km) { return km; };  
} else {  
  operation = function (km) { return km * 0.62; };  
}
```

This conditional assignment is only possible with an expression - a declaration can’t be built conditionally like this in the same way.

5. Quick identification trick

Ask: **does the function keyword start the statement, with nothing before it?**

```
function greet() {} // starts with "function" -> declaration
```

```
const greet = function() {} // starts with "const" -> expression
(function () {})
```

*// starts with "(" -> expression
(specifically an IIFE, covered in Section H)*

6. Summary table

	Function Declaration	Function Expression
Syntax start	function name() {}	const x = function() {}
Needs a name?	Yes	No (can be anonymous)
Hoisted fully?	Yes	No
Usable before its line?	Yes	No
Semicolon at the end?	No	Yes
Good for	Reusable, hoisted utility functions	Conditional assignment, passing as values

7. Practice Questions

Q1. What will this print, and why?

```
console.log(typeof add);

function add(a, b) {
  return a + b;
}
```

Q2. What happens when this runs?

```
multiply(2, 3);

const multiply = function (a, b) {
  return a * b;
};
```

Q3. What happens when this runs?

```
subtract(5, 2);

var subtract = function (a, b) {
  return a - b;
};
```

Q4. Is this a function declaration or a function expression? Explain how you can tell.

```
const calculateArea = function (radius) {
  return 3.14 * radius * radius;
};
```

Q5. Why does this code work, while a similar setup with a function expression would not?

```
start();
```

```
function start() {  
  console.log("Started!");  
}
```

8. Answers

A1. Prints "function". `add` is a function declaration, so it's fully hoisted before `console.log` runs – the whole function (not just its name) is available from the top of the scope.

A2. Throws `ReferenceError`: Cannot access 'multiply' before initialization. `multiply` is declared with `const`, so it's in the Temporal Dead Zone until its line executes – it cannot be called before that line.

A3. Throws `TypeError`: `subtract` is not a function. With `var`, the variable `subtract` is hoisted and exists early, but its value is undefined until the assignment line runs. Calling `undefined(5, 2)` is not valid.

A4. It's a function expression. It starts with `const calculateArea =`, meaning the function keyword appears on the right side of an assignment, not at the very start of the statement.

A5. `start` is a function **declaration**, so JavaScript hoists the entire function (not just a variable) to the top of the scope before execution begins – meaning it can be called on any line, even one written before its definition. A function expression assigned to a variable would not have this full hoisting behavior, as shown in Q2 and Q3.